

ASSIGNMENT 02
Due date: 2026/03/[26|24]

Aim: Activity Lifecycle, Intent Communication and Legacy Layouts

1 The Mother of all Programs: Hello World!

This exercise aims to familiarize you with the project structure and the fundamental differences between declarative and imperative UI design.

Core Concept: UI Paradigms

Modern Android apps use **Jetpack Compose** (declarative), where the UI is described as a function of state. Older projects use **XML Layouts** (imperative), where the UI is defined in static files and manipulated via code.

- ☐ Create a "Hello World" application using the **Empty Activity** template.
- ☐ Create a second project using the **Empty Views Activity** (XML-based).
- ☐ **Deployment:**
 - Make yourself familiar with the generated code stubs, especially with the differences between the two UI approaches.
 - Run your app on an emulated device.
 - (Optional) Run it on your own Android Smartphone (= Physical Device).
- ☐ **Localization:** Add a German translation for your "Hello World" string in `res/values-de/strings.xml` [1].
- ☐ **Analysis:** Investigate and explain the content of your `AndroidManifest.xml` file and answer the following questions:
 - What are the four types of app components and what is their purpose?
 - Does an app need to define one unique entry point or can it have multiple ones? How does the underlying paradigm differ from desktop applications?
 - What is the "application context" and what is the role of the class "R"?

2 Activity Lifecycle

The Lifecycle is the heart of Android's resource management. You must understand how the system manages memory by pausing, stopping, or destroying activities depending on the user's focus and system constraints.

Core Concept: Android Logging Levels

Android provides several logging levels to help you filter messages in **Logcat**. Using the appropriate level is crucial for effective debugging. Use the **Log** class with the following methods:

- **Log.v()** (Verbose): Detailed debug output (lowest priority).
- **Log.d()** (Debug): Used for development and troubleshooting.
- **Log.i()** (Info): General runtime messages.
- **Log.w()** (Warning): Used for potential issues that are not errors.
- **Log.e()** (Error): Serious issues that impact functionality.

- ☐ **Note:** Use the XML-based legacy layout for this exercise.
- ☐ Create two Activities: **MainActivity** and **SecondActivity**.
- ☐ Implement all lifecycle callback methods (**onCreate**, **onStart**, **onResume**, **onPause**, **onStop**, **onDestroy**, **onRestart**) in both [2].
- ☐ **Logging Task:**
 - Use different log levels for different phases. For example, use **Log.d()** for initialization and **Log.i()** for state changes.
 - Explain the difference between these levels in your documentation. Why should you avoid using **Log.v()** or **Log.d()** in a production release?
- ☐ Open the Logcat window in Android Studio, filter by your chosen TAG (e.g., "Lifecycle"), and observe the output.
- ☐ **Experiment:** Describe what happens in the logs during the following events:
 1. Pressing the Home button and then reopening the app.
 2. Rotating the device (Configuration Change).
 3. Navigating from Activity A to B and returning via the Back button.

3 Inter-Activity Communication: Intents

Intents act as the "glue" between components. They allow you to trigger actions and pass data between different parts of your application or even between different apps.

Technical Guide: Intent Types

Explicit Intents specify the exact class (Activity) to be started. This is typically used for internal navigation. **Implicit Intents** declare a general action (e.g., `ACTION_VIEW`) and let the Android system find a suitable handler (e.g., a Browser or Map app) based on the data provided [3].

□ **Note:** Continue using the XML-based legacy layout for this exercise.

□ **Explicit Intent Workflow:**

- In the first Activity, implement a `TextInputEditText` for the user to enter a street address.
- Add a **Button** (e.g., "Send Address") that, when clicked, launches the second Activity and passes the entered string as an "Extra".

□ **Implicit Intent Workflow:**

- In the second Activity, display the received address in a `TextView`.
- Add another **Button** (e.g., "View on Map") that uses an **Implicit Intent** with a `geo: URI` to launch Google Maps and show the location.

□ **Analysis:** Elaborate on the fundamental differences between explicit and implicit intents. In which scenarios is an implicit intent superior to an explicit one?

4 View Binding: Modernizing Legacy UI

To avoid the pitfalls of the traditional `findViewById()` approach—such as `NullPointerException`s and lack of type safety—Android introduced *View Binding*.

Core Concept: How it Works

View Binding is a feature that allows you to more easily write code that interacts with views. Once enabled in a module, it generates a *binding class* for every XML layout file present in that module. An instance of a binding class contains direct references to all views that have an ID in the corresponding layout.

□ **Implementation:**

- Enable `viewBinding` in your `build.gradle.kts` file [4].

- Refactor your "Intents" application from the previous task. Remove all calls to `findViewById()` and use the generated Binding-Class instead.

□ Reflection & Theory:

- Explain the main steps of the implementation (Gradle setup, inflation, root view access).
- Contrast View Binding with `findViewById()`. What are the specific advantages regarding **Type Safety** and **Null Safety**?
- Are there any disadvantages or scenarios where you would still prefer the traditional way?

Bonus: Data Binding

Explore the more advanced **Data Binding Library**.

- What are the main differences between *View Binding* and *Data Binding*?
- Experiment with binding a data object (e.g., a "User" class) directly to your XML layout.
- When would you use which concept?

References

- [1] Google. *Localizing your app*. <https://developer.android.com/training/basics/supporting-devices/languages>
- [2] Google. *The Activity Lifecycle*. <https://developer.android.com/guide/components/activities/activity-lifecycle>
- [3] Google. *Intents and Intent Filters*. <https://developer.android.com/guide/components/intents-filters>
- [4] Google. *View Binding Guide*. <https://developer.android.com/topic/libraries/view-binding>